

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра Информационных систем

Практическая работа

по дисциплине «Теория принятия решений»

Тема: «Применение методов линейного программирования для решения задачи поиска оптимального плана заказа стали на завод»

Студент гр. 3376

Преподаватель

Михайлов Н.Ф.

Пономарёв А.В.

Санкт-Петербург
2026

Содержание

1	Введение	2
2	Постановка задачи	2
3	Формализация задачи	3
3.1	Формальная постановка	3
3.2	Особенность численного решения	4
4	Решение подзадач	4
4.1	Подзадача 1. Базовое решение	4
4.2	Подзадача 2. Анализ начального запаса стали	5
4.3	Подзадача 3. Анализ начальной мощности станкостроительного завода	5
5	Заключение	6
6	Листинг кода	7
6.1	Реализация на Python	7

1 Введение

Целью данной практической работы является освоение метода линейного программирования в задаче оптимального распределения стали между производством, воспроизводством ресурса и расширением производственных мощностей.

Для этого необходимо:

1. формализовать задачу распределения стали по годам;
2. записать переменные, целевую функцию и ограничения;
3. представить динамику запасов и производственных мощностей в виде линейных ограничений;
4. реализовать численное решение задачи линейного программирования;
5. исследовать влияние начальных условий на структуру и результат решения.

2 Постановка задачи

В пункте B_2 сталь поступает на условный производственный комплекс, состоящий из сталелитейного и станкостроительного заводов. Комплекс функционирует в течение 5 лет.

Начальный запас стали составляет 3800 т. Исходные производственные мощности:

- сталелитейный завод — 2500 т стали в год;
- станкостроительный завод — 1700 станков в год.

Сталь может быть распределена между четырьмя направлениями:

- производство станков;
- производство стали;
- расширение мощности сталелитейного завода;
- расширение мощности станкостроительного завода.

Для производства одного станка требуется 1.4 т стали. Каждая тонна стали, направленная на производство стали, обеспечивает выпуск 2.4 т стали. Каждая тонна стали, направленная на расширение сталелитейного завода, увеличивает его мощность на 0.1 т в год. Для увеличения мощности станкостроительного завода на один станок в год требуется 11 т стали.

Решение о распределении стали реализуется в конце очередного года, поэтому увеличенные производственные мощности используются начиная со следующего года. Станкостроительный завод не может получать более половины имеющегося запаса стали.

Требуется:

1. определить план распределения стали, обеспечивающий максимальный суммарный выпуск станков за 5 лет;
2. исследовать влияние начального запаса стали в диапазоне 1000–10000 т на структуру оптимального решения и суммарный выпуск станков;
3. исследовать влияние начальной мощности станкостроительного завода в диапазоне 100–5000 станков в год на структуру оптимального решения и суммарный выпуск станков.

3 Формализация задачи

3.1 Формальная постановка

Переменные.

Плановый горизонт составляет 5 лет. Для каждого года $i = 1, \dots, 5$ вводятся управляющие переменные:

- x_i — количество стали, направляемое на производство станков, т;
- y_i — количество стали, направляемое на производство стали, т;
- z_i — количество стали, направляемое на расширение сталелитейного завода, т;
- w_i — количество стали, направляемое на расширение станкостроительного завода, т.

Также вводятся переменные состояния на начало каждого года:

- s_i — запас стали в начале года i , т;
- p_i^{steel} — мощность сталелитейного завода, т стали в год;
- p_i^{mach} — мощность станкостроительного завода, станков в год.

Состояния задаются для $i = 1, \dots, 6$, так как после пятого года необходимо описать итоговый запас и итоговые мощности.

Цель задачи — максимизировать суммарный выпуск станков за 5 лет:

$$\max Z = \sum_{i=1}^5 \frac{x_i}{1.4}. \quad (1)$$

Начальные условия:

$$s_1 = 3800, \quad (2)$$

$$p_1^{steel} = 2500, \quad (3)$$

$$p_1^{mach} = 1700. \quad (4)$$

Для каждого года $i = 1, \dots, 5$ выполняются следующие ограничения.

Баланс распределения стали:

$$x_i + y_i + z_i + w_i \leq s_i. \quad (5)$$

Станкостроительный завод не может получать более половины имеющегося запаса стали. Так как сталь поступает на него как для выпуска станков, так и для расширения мощности, ограничение имеет вид:

$$x_i + w_i \leq 0.5s_i. \quad (6)$$

Выпуск станков ограничен мощностью станкостроительного завода:

$$\frac{x_i}{1.4} \leq p_i^{mach}. \quad (7)$$

Выпуск стали ограничен мощностью сталелитейного завода:

$$2.4y_i \leq p_i^{steel}. \quad (8)$$

Динамика запаса стали:

$$s_{i+1} = s_i - x_i - y_i - z_i - w_i + 2.4y_i. \quad (9)$$

Или, после приведения подобных слагаемых:

$$s_{i+1} = s_i - x_i + 1.4y_i - z_i - w_i. \quad (10)$$

Динамика мощности сталелитейного завода:

$$p_{i+1}^{steel} = p_i^{steel} + 0.1z_i. \quad (11)$$

Динамика мощности станкостроительного завода:

$$p_{i+1}^{mach} = p_i^{mach} + \frac{w_i}{11}. \quad (12)$$

Все переменные неотрицательны:

$$x_i, y_i, z_i, w_i, s_i, p_i^{steel}, p_i^{mach} \geq 0. \quad (13)$$

3.2 Особенность численного решения

Несмотря на поэтапную структуру задачи, все ограничения и переходы состояния являются линейными. Поэтому на практике задача решается как одна задача линейного программирования, развернутая по всем годам планового периода.

Вектор неизвестных включает управляющие переменные x_i, y_i, z_i, w_i для $i = 1, \dots, 5$, а также переменные состояния $s_i, p_i^{steel}, p_i^{mach}$ для $i = 1, \dots, 6$. После этого целевая функция и все ограничения записываются в матричной форме, пригодной для передачи в LP-решатель.

Такой подход не требует дискретизации состояний и управлений и позволяет получить точное решение линейной модели.

4 Решение подзадач

4.1 Подзадача 1. Базовое решение

Для исходных значений $s_1 = 3800, p_1^{steel} = 2500, p_1^{mach} = 1700$ получен следующий оптимальный план.

Год	s_i	p_i^{steel}	p_i^{mach}	x_i	y_i	z_i	w_i	Выпуск
1	3800.00	2500.00	1700.00	1900.00	1041.67	0.00	0.00	1357.14
2	3358.33	2500.00	1700.00	1679.17	1041.67	0.00	0.00	1199.40
3	3137.50	2500.00	1700.00	1568.75	1041.67	0.00	0.00	1120.54
4	3027.08	2500.00	1700.00	1513.54	1041.67	0.00	0.00	1081.10
5	2971.88	2500.00	1700.00	1485.94	0.00	0.00	0.00	1061.38

Суммарный выпуск за 5 лет составил:

$$Z = 5819.57 \text{ станков.}$$

Итоговое состояние комплекса:

$$s_6 = 1485.94, \quad p_6^{steel} = 2500, \quad p_6^{mach} = 1700.$$

Вывод.

В базовом решении расширение мощностей не используется. В первые четыре года сталелитейный завод работает на пределе мощности:

$$2.4 \cdot 1041.67 \approx 2500.$$

Кроме того, в каждом году на производство станков направляется половина имеющегося запаса стали. Следовательно, основными ограничивающими факторами являются запас стали и мощность сталелитейного завода, а начальная мощность станкостроительного завода для базового варианта достаточна.

4.2 Подзадача 2. Анализ начального запаса стали

Проводилось изменение начального запаса стали в диапазоне от 1000 до 10000 т. Остальные начальные параметры оставались неизменными:

$$p_1^{steel} = 2500, \quad p_1^{mach} = 1700.$$

Нач. запас	Выпуск	x_1	y_1	z_1	w_1	s_6	p_6^{steel}	p_6^{mach}
1000	3820.68	0.00	1000.00	0.00	0.00	1426.04	2500.00	1700.00
2000	4572.17	958.33	1041.67	0.00	0.00	1432.29	2500.00	1700.00
3000	5266.00	1500.00	1041.67	0.00	0.00	1460.94	2500.00	1700.00
4000	5957.96	2000.00	1041.67	0.00	0.00	1492.19	2500.00	1700.00
5000	6644.57	2380.00	1041.67	0.00	0.00	1530.94	2500.00	1700.00
6000	7300.00	2380.00	1041.67	0.00	0.00	1613.33	2500.00	1700.00
7000	7889.58	2380.00	1041.67	0.00	0.00	1787.92	2500.00	1700.00
8000	8340.48	2380.00	1041.67	0.00	0.00	2156.67	2500.00	1700.00
9000	8622.96	2380.00	1041.67	0.00	338.15	2423.04	2500.00	1730.74
10000	8845.19	2380.00	1041.67	0.00	949.26	2500.81	2500.00	1786.30

Вывод.

При малом начальном запасе стали оптимальный план направляет значительную часть ресурса на производство стали. Например, при запасе 1000 т в первый год весь ресурс направляется на производство стали, а выпуск станков в первый год отсутствует.

При увеличении начального запаса суммарный выпуск возрастает. Начиная с запаса 5000 т величина x_1 достигает 2380 т, что соответствует ограничению мощности станкостроительного завода:

$$1700 \cdot 1.4 = 2380.$$

Дальнейший рост начального запаса уже не может увеличить выпуск первого года за счет x_1 , поэтому при больших запасах появляется расширение станкостроительного завода ($w_1 > 0$).

4.3 Подзадача 3. Анализ начальной мощности станкостроительного завода

Проводилось изменение начальной мощности станкостроительного завода в диапазоне от 100 до 5000 станков в год. Начальный запас стали и мощность сталелитейного завода оставались равными:

$$s_1 = 3800, \quad p_1^{steel} = 2500.$$

p_1^{mach}	Выпуск	x_1	y_1	z_1	w_1	s_6	p_6^{steel}	p_6^{mach}
100	1764.58	140.00	1041.67	0.00	1760.00	1126.26	2554.60	551.15
600	3704.18	840.00	1041.67	0.00	1060.00	1327.87	2521.75	841.21
1100	5587.41	1540.00	1041.67	0.00	240.37	1570.59	2500.00	1121.85
1600	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	1600.00
2100	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	2100.00
2600	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	2600.00
3100	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	3100.00
3600	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	3600.00
4100	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	4100.00
4600	5819.57	1900.00	1041.67	0.00	0.00	1485.94	2500.00	4600.00

Вывод.

При малой начальной мощности станкостроительного завода существенная часть стали направляется на его расширение. Так, при мощности 100 станков в год в первый год на расширение направляется 1760 т стали. Это позволяет увеличить выпуск в последующие годы.

При росте начальной мощности суммарный выпуск увеличивается до значения 5819.57 станков. Начиная примерно с мощности 1600 станков в год дальнейшее увеличение начальной мощности уже не повышает результат. Это означает, что после этого порога ограничивающим фактором становится не станкостроительный завод, а запас стали и мощность сталелитейного завода.

5 Заключение

В ходе работы задача распределения стали между производством станков, производством стали и расширением мощностей была сведена к задаче линейного программирования. Несмотря на поэтапную структуру исходной постановки, линейность ограничений и переходов состояния позволяет рассматривать весь пятилетний период как единую LP-модель.

В базовом варианте суммарный выпуск составил 5819.57 станков. Расширение производственных мощностей в оптимальном плане не используется, так как при исходных параметрах основной эффект достигается за счет распределения стали между выпуском станков и производством новой стали.

Исследование начального запаса стали показало, что рост запаса увеличивает суммарный выпуск, однако после достижения ограничения по мощности станкостроительного завода дополнительная сталь начинает использоваться для расширения этой мощности. Исследование начальной мощности станкостроительного завода показало наличие порогового значения: начиная примерно с 1600 станков в год дальнейшее увеличение мощности не повышает суммарный выпуск, поскольку ограничивающим фактором становится ресурсная база и мощность сталелитейного завода.

Таким образом, метод линейного программирования позволяет не только найти оптимальный пятилетний план распределения стали, но и исследовать влияние начальных условий на структуру решения.

6 Листинг кода

6.1 Реализация на Python

```
1 from __future__ import annotations
2
3 import numpy as np
4 from cvxopt import matrix, solvers
5
6 try:
7     from tabulate import tabulate
8 except ModuleNotFoundError:
9     tabulate = None
10
11 solvers.options["show_progress"] = False
12
13
14 YEARS = 5
15
16 DEFAULT_INITIAL_STEEL = 3800
17 DEFAULT_STEEL_CAPACITY = 2500
18 DEFAULT_MACHINE_CAPACITY = 1700
19
20 STEEL_PER_MACHINE = 1.4
21 STEEL_OUTPUT_RATE = 2.4
22 STEEL_CAPACITY_GROWTH = 0.1
23 MACHINE_CAPACITY_COST = 11
24
25 CONTROL_BLOCK = YEARS
26 STATE_BLOCK = YEARS + 1
27 NUM_VARIABLES = 4 * CONTROL_BLOCK + 3 * STATE_BLOCK
28
29
30 def x_idx(year: int) -> int:
31     return year - 1
32
33
34 def y_idx(year: int) -> int:
35     return CONTROL_BLOCK + year - 1
36
37
38 def z_idx(year: int) -> int:
39     return 2 * CONTROL_BLOCK + year - 1
40
41
42 def w_idx(year: int) -> int:
43     return 3 * CONTROL_BLOCK + year - 1
44
45
46 def s_idx(year: int) -> int:
47     return 4 * CONTROL_BLOCK + year - 1
48
49
```



```

50 def steel_cap_idx(year: int) -> int:
51     return 4 * CONTROL_BLOCK + STATE_BLOCK + year - 1
52
53
54 def machine_cap_idx(year: int) -> int:
55     return 4 * CONTROL_BLOCK + 2 * STATE_BLOCK + year - 1
56
57
58 def empty_row() -> list[float]:
59     return [0.0] * NUM_VARIABLES
60
61
62 def add_ub(G: list[list[float]], h: list[float], row: list[float], rhs: float)
    -> None:
63     G.append(row)
64     h.append(rhs)
65
66
67 def add_eq(A: list[list[float]], b: list[float], row: list[float], rhs: float)
    -> None:
68     A.append(row)
69     b.append(rhs)
70
71
72 def solve_problem(
73     initial_steel: float = DEFAULT_INITIAL_STEEL,
74     initial_steel_capacity: float = DEFAULT_STEEL_CAPACITY,
75     initial_machine_capacity: float = DEFAULT_MACHINE_CAPACITY,
76 ) -> tuple[np.ndarray, float]:
77     c = empty_row()
78
79     for year in range(1, YEARS + 1):
80         c[x_idx(year)] = -1.0 / STEEL_PER_MACHINE
81
82     G: list[list[float]] = []
83     h: list[float] = []
84     A: list[list[float]] = []
85     b: list[float] = []
86
87     row = empty_row()
88     row[s_idx(1)] = 1
89     add_eq(A, b, row, initial_steel)
90
91     row = empty_row()
92     row[steel_cap_idx(1)] = 1
93     add_eq(A, b, row, initial_steel_capacity)
94
95     row = empty_row()
96     row[machine_cap_idx(1)] = 1
97     add_eq(A, b, row, initial_machine_capacity)
98
99     for year in range(1, YEARS + 1):
100         row = empty_row()

```

```

101     row[x_idx(year)] = 1
102     row[y_idx(year)] = 1
103     row[z_idx(year)] = 1
104     row[w_idx(year)] = 1
105     row[s_idx(year)] = -1
106     add_ub(G, h, row, 0)
107
108     row = empty_row()
109     row[x_idx(year)] = 1
110     row[w_idx(year)] = 1
111     row[s_idx(year)] = -0.5
112     add_ub(G, h, row, 0)
113
114     row = empty_row()
115     row[x_idx(year)] = 1 / STEEL_PER_MACHINE
116     row[machine_cap_idx(year)] = -1
117     add_ub(G, h, row, 0)
118
119     row = empty_row()
120     row[y_idx(year)] = STEEL_OUTPUT_RATE
121     row[steel_cap_idx(year)] = -1
122     add_ub(G, h, row, 0)
123
124     row = empty_row()
125     row[s_idx(year + 1)] = 1
126     row[s_idx(year)] = -1
127     row[x_idx(year)] = 1
128     row[y_idx(year)] = -(STEEL_OUTPUT_RATE - 1)
129     row[z_idx(year)] = 1
130     row[w_idx(year)] = 1
131     add_eq(A, b, row, 0)
132
133     row = empty_row()
134     row[steel_cap_idx(year + 1)] = 1
135     row[steel_cap_idx(year)] = -1
136     row[z_idx(year)] = -STEEL_CAPACITY_GROWTH
137     add_eq(A, b, row, 0)
138
139     row = empty_row()
140     row[machine_cap_idx(year + 1)] = 1
141     row[machine_cap_idx(year)] = -1
142     row[w_idx(year)] = -1 / MACHINE_CAPACITY_COST
143     add_eq(A, b, row, 0)
144
145     for variable_index in (y_idx(YEARS), z_idx(YEARS), w_idx(YEARS)):
146         row = empty_row()
147         row[variable_index] = 1
148         add_eq(A, b, row, 0)
149
150     for variable in range(NUM_VARIABLES):
151         row = empty_row()
152         row[variable] = -1
153         add_ub(G, h, row, 0)

```

```

154
155     sol = solvers.lp(
156         matrix(c, tc="d"),
157         matrix(np.array(G, dtype=float), tc="d"),
158         matrix(np.array(h, dtype=float), tc="d"),
159         matrix(np.array(A, dtype=float), tc="d"),
160         matrix(np.array(b, dtype=float), tc="d"),
161     )
162
163     solution = np.array(sol["x"]).flatten()
164     max_machines = -float(sol["primal objective"])
165     return solution, max_machines
166
167
168 def format_number(value: float) -> str:
169     if abs(value) < 1e-5:
170         value = 0.0
171     return f"{value:.2f}"
172
173
174 def print_table(headers: list[str], rows: list[list[object]]) -> None:
175     string_rows = [[str(cell) for cell in row] for row in rows]
176
177     if tabulate is not None:
178         print(
179             tabulate(
180                 string_rows,
181                 headers=headers,
182                 tablefmt="grid",
183                 stralign="right",
184                 disable_numparse=True,
185             )
186         )
187     return
188
189     table = [headers] + string_rows
190     widths = [max(len(row[column]) for row in table) for column in range(len(
191         headers))]
192
193     def format_row(row: list[str]) -> str:
194         return " | ".join(value.rjust(widths[index]) for index, value in
195             enumerate(row))
196
197     separator = "+-+-.join("-" * width for width in widths)
198     print(format_row(headers))
199     print(separator)
200     for row in string_rows:
201         print(format_row(row))
202
203 def get_year_values(solution: np.ndarray, year: int) -> dict[str, float]:
204     return {
205         "x": solution[x_idx(year)],

```

```

205         "y": solution[y_idx(year)],
206         "z": solution[z_idx(year)],
207         "w": solution[w_idx(year)],
208         "s": solution[s_idx(year)],
209         "steel_cap": solution[steel_cap_idx(year)],
210         "machine_cap": solution[machine_cap_idx(year)],
211         "machines": solution[x_idx(year)] / STEEL_PER_MACHINE,
212     }
213
214
215 def print_solution(solution: np.ndarray, max_machines: float) -> None:
216     print("\n===== OPTIMAL PLAN =====")
217     headers = [
218         "Year",
219         "Steel",
220         "Steel cap",
221         "Machine cap",
222         "x",
223         "y",
224         "z",
225         "w",
226         "Output",
227     ]
228     rows = []
229
230     for year in range(1, YEARS + 1):
231         values = get_year_values(solution, year)
232         rows.append(
233             [
234                 year,
235                 format_number(values["s"]),
236                 format_number(values["steel_cap"]),
237                 format_number(values["machine_cap"]),
238                 format_number(values["x"]),
239                 format_number(values["y"]),
240                 format_number(values["z"]),
241                 format_number(values["w"]),
242                 format_number(values["machines"]),
243             ]
244         )
245
246     print_table(headers, rows)
247     print(f"Total output: {max_machines:.2f} machines")
248
249
250 def task1_base_plan() -> None:
251     print("=== TASK 1. BASE PLAN ===")
252     solution, max_machines = solve_problem()
253     print_solution(solution, max_machines)
254
255
256 def task2_research_initial_steel() -> None:
257     print("\n=== TASK 2. INITIAL STEEL ANALYSIS ===")

```

```

258
259 headers = ["Initial steel", "Output", "x1", "y1", "z1", "w1", "Final steel"]
260 rows = []
261
262 for initial_steel in range(1000, 10001, 1000):
263     solution, max_machines = solve_problem(initial_steel=initial_steel)
264     first_year = get_year_values(solution, 1)
265     rows.append(
266         [
267             initial_steel,
268             format_number(max_machines),
269             format_number(first_year["x"]),
270             format_number(first_year["y"]),
271             format_number(first_year["z"]),
272             format_number(first_year["w"]),
273             format_number(solution[s_idx(YEARS + 1)]),
274         ]
275     )
276
277 print_table(headers, rows)
278
279
280 def task3_research_machine_capacity() -> None:
281     print("\n=== TASK 3. MACHINE CAPACITY ANALYSIS ===")
282
283     headers = ["Initial machine cap", "Output", "x1", "y1", "z1", "w1", "Final
284 cap"]
285     rows = []
286
287     machine_capacity_values = [100] + list(range(500, 5001, 500))
288
289     for machine_capacity in machine_capacity_values:
290         solution, max_machines = solve_problem(initial_machine_capacity=
291 machine_capacity)
292         first_year = get_year_values(solution, 1)
293         rows.append(
294             [
295                 machine_capacity,
296                 format_number(max_machines),
297                 format_number(first_year["x"]),
298                 format_number(first_year["y"]),
299                 format_number(first_year["z"]),
300                 format_number(first_year["w"]),
301                 format_number(solution[machine_cap_idx(YEARS + 1)]),
302             ]
303         )
304
305     print_table(headers, rows)
306
307
308 def main() -> None:
309     task1_base_plan()
310     task2_research_initial_steel()

```

```
309     task3_research_machine_capacity()
310
311
312 if __name__ == "__main__":
313     main()
```